Architectural Governance and Ledger Implementation: A Comprehensive Report on Proof of Work, Archival Mechanics, and Surface Hardening

1. Executive Architecture Overview and Governance Paradigm

The modern software repository is no longer merely a storage facility for code; it has evolved into a complex, agent-interactive environment where human operators and artificial intelligence agents collaborate in real-time. In this context, the governance of "state" and "history" becomes the primary determinant of system integrity. The Stela repository's existing governance framework, anchored by the Policy of Truth (PoT.md), establishes a rigorous constitutional order. However, the current mechanisms for tracking execution fidelity and managing historical drift require significant architectural maturation to meet the demands of a "Pointer-first" constitution.[1] This report investigates and mandates the implementation of a 'Proof of Work' (PoW.md) root file as a persistent, high-integrity ledger, functioning in parallel with the existing 'State of Play' (SoP.md).[1]

The transition from ephemeral "Action Logs" to a persistent PoW.md ledger represents a fundamental shift in the repository's operational philosophy. It moves the system from a model of "trust" to a model of "verification," where every unit of work—defined as a Dispatch Packet (DP)—must generate a cryptographic-grade entry in the root ledger. This entry serves not only as a historical record but as a functional constraint on future agent behavior, preventing the "hallucinated agency" that occurs when AI actors lack a verifiable history of their own actions.[2]

Furthermore, the integrity of this ledgering system relies on strict lifecycle management. Infinite growth of historical records leads to "Context Fatigue," where the relevant signal for an AI agent is drowned out by the noise of obsolete entries. To mitigate this, a strict 30-entry archive policy is designed, enforced by a refactored ops/bin/prune binary.[1] This policy ensures that the active context remains lean and performant while preserving deep history in an immutable archive.

Parallel to the ledger reform, the security of the 'TASK template'—the generative seed for all active work—must be hardened. The current reliance on hidden or git-ignored local templates introduces "Ghost Canon" risks, where the actual structure of work diverges from the documented specification. The solution proposed herein creates a retrieval-based storage

strategy that embeds the protected template directly within the canonical specification docs/ops/specs/surfaces/task.md, enforcing a Single Source of Truth (SSoT) for both the definition and instantiation of work.[5]

Finally, the governance layer requires a linguistic and logic hardening. The ambiguous term "Living Document" is purged in favor of precise terminology that reflects the immutable nature of the repository's truth. Concurrently, the "Mandatory Closing Block" of the DP is subjected to granular validation rules that permit rich expression (Markdown) only where semantically appropriate, while enforcing strict machine-readability elsewhere. These changes are underpinned by a "Dry Run" policing strategy for all destructive operations, ensuring that the system allows for simulation and impact analysis before any irreversible state change occurs.[7]

2. The Proof of Work (PoW.md) Ledger: Theoretical and Structural Analysis

2.1 The Necessity of a Forensic Root Ledger

The existing SoP.md file serves as the "History Ledger" and "shipment record," providing the narrative context of *what* changed and *why*.[1] However, SoP.md is insufficient for capturing the granular, quantitative verification of *how* the work was executed. In an agentic workflow, the "how" is critical for preventing drift. Agents executing tasks need to verify that previous actions resulted in the expected state changes before proceeding. The State of Action (SoA) document [1] currently captures this data but exists as a transient artifact rather than a root-level constitutional surface.

The PoW.md ledger addresses this gap by elevating the forensic logic of the SoA into the repository's root. It acts as the "execution ledger," distinct from the "strategic ledger" (SoP). While SoP records the intent and the outcome, PoW records the *delta*—the precise file modifications, byte counts, and logic conflict resolutions that occurred during execution. This bifurcation enables a dual-layer audit system: human operators review SoP for strategic alignment, while automated linters and agents review PoW for structural integrity.[1]

2.2 Ledger Schema and Components

To function as a persistent root ledger, PoW.md must adhere to a strict schema that facilitates both human readability and machine parsing. The analysis of the State of Action template [1] suggests the following core components for the PoW.md schema:

Table 1: Comparative Taxonomy of Ledger Components

Component	SoP.md	PoW.md	Data Type	Usage in

	(Strategic Ledger)	(Forensic Ledger)		Audit
Identifier	Date/Time + DP ID	Packet ID + Timestamp	Alphanumeric	Indexing key for retrieval.
Agent	Implicit (Author)	Explicit "Executing Agent"	String	Accountability tracing.
Reference	Narrative Summary	Pointer to TASK.md Coord.	Hyperlink	Context rehydration.
Metrics	Qualitative	"Deviation Detected" (%)	Float/Percentage	Automated drift detection.
Impact	"Objective"	"File Modifications" (Bytes)	Diff-Stat	Blast radius analysis.
State	n/a	Equilibrium / Drift	Enum	Gating for future tasks.

2.2.1 Action Identification The entry must begin with a unique Packet ID and ISO 8601 Timestamp.[1] Unlike SoP.md, which is organized by release or shipment, PoW.md is organized by *execution events*. This granularity allows for the tracking of "micro-transactions" of work that may not result in a strategic shipment but still alter the repository state. The "Reference Point" field links the execution directly to a specific coordinate in TASK.md or PoT.md, establishing a hyperlink between the action and the policy it fulfills.

2.2.2 Quantitative Execution Log Adhering to the "Behavioral Logic Standard" defined in PoT.md [1], this section records the sequence of operations with "absolute literalism." Contractions are strictly forbidden to ensure linguistic precision. Each operation is logged with a "Deviation Detected" metric (e.g., "0.0%" for a perfect match). This quantitative approach allows for automated "Drift Assessment," where a linter can parse the ledger and flag any entry where the resulting state is not "Equilibrium".[1]

2.2.3 Structural Impact and Conflict Resolution The "Structural Impact Report" details the exact file modifications, including byte count changes (e.g., +412 bytes). Crucially, it

documents "Logic Conflict Resolution"—instances where the agent encountered an invariant in PoT.md (formerly TRUTH.md) and had to make a deterministic choice to resolve it. This explicit recording of conflict resolution provides the "explainability" required for high-trust agent workflows.[11]

2.3 Integration with the Filing Doctrine and Read-in Order

The placement of PoW.md at the repository root is a deliberate architectural decision. Under the Filing Doctrine [1], the root is reserved for "Canon Surfaces." By placing PoW.md here, it inherits the constitutional protection of PoT.md.

In the "Source of Truth Read-in Order" [1], PoW.md serves as a critical bridge. The updated read-in order for agents is:

1. **PoT.md**: The Constitution (Immutable Laws).
2. **SoP.md**: The History (Context and Intent).
3. **PoW.md**: The Evidence (Forensic State Verification).
4. **TASK.md**: The Assignment (Active Work).

This sequence ensures that an agent verifies the *proof* of the previous state (PoW.md) before attempting to modify the *current* state (TASK.md). This "Proof-before-Task" logic creates a safety interlock that prevents agents from compounding errors based on unverified assumptions.[12]

3. The Archival Engine: ops/bin/prune and the 30-Entry Policy

3.1 The Context Fatigue Problem

Repositories that rely on flat files for history inevitably face "Context Fatigue." As SoP.md and PoW.md grow, they consume an increasing percentage of the AI agent's context window with stale data. This degradation in signal-to-noise ratio leads to "hallucinations" where agents conflate historical patterns with current requirements.[3] The "30-entry archive policy" is a hard constraint designed to maintain the "Freshness" of the context. By limiting the active ledger to the 30 most recent entries, the system ensures that the agent focuses on the immediate operational horizon while retaining access to deep history through archival pointers.[4]

3.2 Analysis of ops/bin/prune Implementation

The ops/bin/prune binary is the enforcement mechanism for this policy. Based on the functional receipts in SoP.md [1], this script already manages SoP.md archival. To support the dual-ledger architecture, it must be refactored to handle PoW.md with identical rigor.

3.2.1 Current Mechanics and Refactoring

The script currently operates by:

1. Scanning the target file (SoP.md) for entry headers.
2. Counting the entries.
3. Calculating the "cut line" (the line number of the 31st entry).
4. Moving the content below the cut line to a new file in storage/archives/root/.
5. Rewriting the target file with only the header and the first 30 entries.

For the PoW.md implementation, the script must abstract the "Target" into a variable. The logic must distinguish between the "Strategic Archive" (SoP-archive-YYYY-MM.md) and the "Forensic Archive" (PoW-archive-YYYY-MM.md). This separation is vital because the *frequency* of PoW entries (one per execution) will likely exceed SoP entries (one per shipment), leading to different archival velocities.[9]

3.2.2 The "Uncommitted Results" Guard A critical safety feature of ops/bin/prune is the "uncommitted RESULTS guard" mentioned in the ledger history.[1] Before pruning PoW.md (which links to result proofs), the script must verify that the corresponding result artifacts in storage/handoff/ have been committed to git. Pruning a PoW entry that points to an uncommitted or lost verification receipt breaks the chain of custody. The script must execute a git status --porcelain check on the referenced artifacts. If any artifact is untracked or modified, the prune operation must strictly abort with a SAFETY VIOLATION error.[1]

3.2.3 Archive Naming and Retrieval The archive naming convention follows the ISO-8601 standard (YYYY-MM) to facilitate chronological sorting. The ops/bin/prune script ensures that archives are created in storage/archives/root/.[1] This directory is strictly defined as "Trash" or "Museum" storage in the Filing Doctrine, meaning it is available for reference but is not part of the active "Run" context. This distinction allows the ops/bin/dump tool to exclude these archives from the standard context bundle, thereby enforcing the "lean context" policy.[16]

4. The Protected TASK Template: Embedding and Retrieval

4.1 The "Ghost Canon" Risk

In previous iterations, the TASK.md template (formerly "Golden Master") was stored as a local, git-ignored file or within a hidden directory. This created a "Ghost Canon" risk: the template used to initialize work was not visible in the repository's tracked documentation. If the template evolved locally on an operator's machine, the centralized documentation (docs/) became stale, leading to "drift" between the documented specification and the actual work surface.[1]

4.2 Embedding Strategy: The Spec as the Container

To resolve this, the repository adopts an "Embedded Spec" strategy. The TASK.md template is

moved directly into the docs/ops/specs/surfaces/task.md file. This file serves a dual purpose:

1. **Documentation**: It explains the rationale, schema, and rules of the TASK surface.
2. **Container**: It physically holds the raw markdown of the template within designated delimiters.

This approach ensures that the *specification* and the *template* are atomically linked. It is impossible to update the template without updating the spec, enforcing the "Truth" of the document.

4.2.1 Delimiter Design To facilitate robust extraction by shell scripts (like sed or awk), the template is enclosed in unique, non-colliding delimiters. We utilize HTML comment-style markers that are invisible in rendered markdown but easily parseable by regex.[18]

Structure:

STELA TASK DASHBOARD

Status: ACTIVE

...

These delimiters are chosen because sed can reliably identify ranges between patterns (/START/,/END/). By using HTML comments, the template remains readable in the GitHub UI or any markdown viewer, allowing human operators to manually copy it if necessary, while maintaining machine readability.[20]

4.3 Extraction Logic (The "Retrieval" Mechanism)

The extraction mechanism is integrated into the ops/bin/open or project initialization scripts. Instead of copying a file (cp templates/TASK.md TASK.md), the script "hydrates" the TASK.md file by extracting the content from the spec.

Technique:

The extraction uses a sed command to:

1. Read docs/ops/specs/surfaces/task.md.
2. Locate the range between and.
3. Exclude the delimiters themselves (to prevent nested comments in the generated file).
4. Write the output to TASK.md.

This "Hydration" process ensures that every new task session begins with the exact version of the template defined in the canonical specification, effectively eliminating the risk of drift from local, stale copies.[18]

5. Governance Hardening: Terminology, Validation, and Policing

5.1 The "Living Document" Terminology Scrub

The phrase "Living Document" has been identified as a vector for semantic drift. In the context of a "Proof of Work" system, a document is not "living"—it is either **Mutable** (Active) or **Immutable** (Archived). "Living" implies a lack of version control and accountability. Therefore, a terminology scrub is mandated to replace all instances of "Living Document" or "Living Work Log" with precise terms like "Active Surface" or "Execution Log".[2]

This scrub is enforced by the tools/lint/truth.sh script. This linter currently scans for "forbidden spellings" (like typos of "Stela"). It is updated to include "Living Document" as a forbidden phrase. Any Pull Request containing this phrase in docs/ or ops/ will fail the "Truth Lint" gate, ensuring linguistic purity is maintained across the repository.[1]

5.2 Mandatory Closing Block Enforcement

The "Mandatory Closing Block" in TASK.md is the final handshake of the Dispatch Packet. It summarizes the work for the changelog and git history. The validation of this block requires a nuanced approach because different fields have different data type requirements.[1]

5.2.1 Field-Specific Validation Logic

The tools/lint/dp.sh script is responsible for validating the DP. It must be enhanced to parse the Closing Block and apply specific regex rules to each field:

- **Field 1 (Primary Commit Header)**, **Field 2 (Pull Request Title)**, **Field 4 (Final Squash Stub)**, **Field 5 (Extended Technical Manifest)**: These fields must be **Strict Plaintext**. They are often used in git commit messages or filenames, where markdown characters (like backticks or asterisks) can cause shell expansion errors or formatting issues. The validation logic checks for the presence of forbidden characters (*, _, `, [,]) and fails if found.[24]
- **Field 3 (Pull Request Description)** and **Field 6 (Review Conversation Starter)**: These fields are intended for human consumption on platforms like GitHub. They benefit from rich text (lists, code blocks). The validation logic for these fields is **Permissive**, checking only for existence (non-empty) and potentially checking for "placeholder" values (like "ENTER DESCRIPTION HERE"), which are rejected.[24]

This granular enforcement ensures that the "Receipt" generated at the end of a task is both machine-safe (for git operations) and human-rich (for review context).

5.3 The "Dry Run" Policing Strategy

Destructive operations—specifically pruning ledgers and archiving files—carry a high risk of

data loss. To mitigate this, a "Dry Run" policing strategy is implemented for ops/bin/prune and related maintenance binaries.

5.3.1 Simulation Mode All maintenance scripts must support a --dry-run flag. When this flag is active, the script performs all calculations (identifying files, counting entries, determining cut lines) but *skips* the actual I/O operations (file moves, deletions, overwrites). Instead, it outputs a detailed report of the *intended* actions.[7]

5.3.2 CI/CD Integration

The repo-gates workflow utilizes this "Dry Run" capability. During a PR build, the CI executes ops/bin/prune --dry-run. If the dry run indicates that a prune *would* occur (e.g., the ledger is over 30 entries), the CI can either:

1. **Warn**: Notify the operator that a prune will happen upon merge/deployment.
2. **Fail (Strict Mode)**: Require the operator to manually run the prune and commit the result, ensuring that the archival action is explicit and reviewed rather than an automated side effect.[8]

This strategy ensures that the "State of Play" and "Proof of Work" ledgers are never modified silently by automation, adhering to the "Operator Authority" principle defined in the Staffing Protocol.[1]

6. Implementation Solutions and Technical Specifications

The following solutions provide the specific code and logic required to implement the architecture described above.

SOLUTION 1: ops/bin/prune Refactor for Dual-Ledger Archiving

Objective: Refactor ops/bin/prune to support PoW.md alongside SoP.md with a 30-entry limit, utilizing the "Dry Run" strategy.

Implementation Script:

Bash

```
#!/usr/bin/env bash
# Prune: Dual-Ledger Archival Tool
# Enforces 30-entry limit on SoP.md and PoW.md
# Usage:./ops/bin/prune --target=[sop|pow][--dry-run]
```

```
set -euo pipefail

# Default values
TARGET="sop"
DRY_RUN=0
REPO_ROOT="$(git rev-parse --show-toplevel)"
ARCHIVE_ROOT="${REPO_ROOT}/storage/archives/root"
MAX_ENTRIES=30

# Argument Parsing
for arg in "$@"; do
  case $arg in
    --target=*)
      TARGET="${arg#*=}"
      ;;
    --dry-run)
      DRY_RUN=1
      ;;
    *)
      echo "Error: Unknown argument $arg"
      exit 1
      ;;
  esac
done

# Configuration based on target
if]; then
    LEDGER_FILE="SoP.md"
    ARCHIVE_PREFIX="SoP"
elif]; then
    LEDGER_FILE="PoW.md"
    ARCHIVE_PREFIX="PoW"
else
    echo "Error: Invalid target '$TARGET'. Use 'sop' or 'pow'."
    exit 1
fi

LEDGER_PATH="${REPO_ROOT}/${LEDGER_FILE}"

# 1. Verification
if]; then
    echo "Error: ${LEDGER_FILE} not found at $LEDGER_PATH"
```

```
    exit 1
fi

# 2. Entry Counting
# Assumes entries start with '## YYYY-MM-DD' or similar Level 2 header
ENTRY_COUNT=$(grep -c "^## [0-9]\{4\}" "$LEDGER_PATH" |

| true)

if; then
    echo "OK: $LEDGER_FILE has $ENTRY_COUNT entries. No pruning required."
    exit 0
fi

# 3. Pruning Logic
PRUNE_COUNT=$((ENTRY_COUNT - MAX_ENTRIES))
# Identify the line number of the 31st entry (The Cut Line)
CUT_LINE=$(grep -n "^## [0-9]\{4\}" "$LEDGER_PATH" | sed -n "$((MAX_ENTRIES + 1))p" | cut -d:
-f1)

ARCHIVE_NAME="${ARCHIVE_PREFIX}-archive-$(date +%Y-%m).md"
ARCHIVE_PATH="${ARCHIVE_ROOT}/${ARCHIVE_NAME}"

# 4. Dry Run / Execution
if; then
    echo " Analysis for $LEDGER_FILE:"
    echo "  - Total Entries: $ENTRY_COUNT"
    echo "  - To Prune: $PRUNE_COUNT"
    echo "  - Cut Line: $CUT_LINE"
    echo "  - Archive Destination: $ARCHIVE_PATH"
    echo "  - Action: Would move lines $CUT_LINE+ to archive."
    exit 0
fi

echo "Pruning $PRUNE_COUNT entries from $LEDGER_FILE..."

# Ensure archive directory exists
mkdir -p "$ARCHIVE_ROOT"

# Atomic Operation:
# A. Extract archive content to temp file
# B. Extract keep content to temp file
# C. Append archive content to actual archive file
# D. Overwrite ledger with keep content
```

```
TMP_ARCHIVE=$(mktemp)
TMP_KEEP=$(mktemp)

# Extract content
tail -n +$CUT_LINE "$LEDGER_PATH" >> "$TMP_ARCHIVE"
head -n $((CUT_LINE - 1)) "$LEDGER_PATH" > "$TMP_KEEP"

# Write to Archive (Append mode)
cat "$TMP_ARCHIVE" >> "$ARCHIVE_PATH"

# Overwrite Ledger
mv "$TMP_KEEP" "$LEDGER_PATH"
rm -f "$TMP_ARCHIVE"

echo "SUCCESS: Archived to $ARCHIVE_PATH"
```

SOLUTION 2: Protected TASK Template Embedding and Extraction

Objective: Embed the TASK.md template into docs/ops/specs/surfaces/task.md and provide the extraction command for ops/bin/open.

Specification File Content (docs/ops/specs/surfaces/task.md):

TASK Surface Specification

1. Overview

The TASK.md file is the active work surface...

2. Template Structure

The following block defines the canonical structure of a TASK file.

It is protected by the repo-gates workflow.

STELA TASK DASHBOARD

Status: ACTIVE

Owner: Integrator

Last Updated: 2026-02-13

1. Session State

Pointer: (Dynamic)

Context Manifest: ops/lib/manifests/CONTEXT.md

2. Logic Pointers

... (Rest of Template)...

Extraction Command (Integration for ops/bin/open):

Bash

```
# Extract TASK template from spec
# Logic: Find range between TEMPLATE-START/END, delete the delimiter lines, and write to TASK.md
sed -n '//,//{
 /
```

Works cited

1. dump-full-main-2dafc32f.txt
2. AI Guardrails: Tutorial & Best Practices - Patronus AI, accessed February 13, 2026, https://www.patronus.ai/ai-reliability/ai-guardrails
3. Preventing AI Mistakes in Production: Inside Cleanlab's Guardrails, accessed February 13, 2026, https://cleanlab.ai/blog/inside-trustworthiness-guardrail/
4. git-archive Documentation - Git, accessed February 13, 2026, https://git-scm.com/docs/git-archive
5. Markdown : A Smarter choice for Embeddings Than JSON or XML | by kanishk khatter, accessed February 13, 2026, https://medium.com/@kanishk.khatter/markdown-a-smarter-choice-for-embeddings-than-json-or-xml-70791ece24df
6. Diving Into Spec-Driven Development With GitHub Spec Kit - Microsoft for Developers, accessed February 13, 2026, https://developer.microsoft.com/blog/spec-driven-development-spec-kit
7. How to Test AI Agent Guardrails: A Complete Framework for Safety, Security, and Compliance | ZioSec, accessed February 13, 2026, https://ziosec.com/article?slug=how-to-test-ai-agent-guardrails
8. AI Red Teaming Agent (preview) - Microsoft Foundry, accessed February 13, 2026, https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/ai-red-teaming-agent?view=foundry-classic
9. Using Blockchain Ledgers to Record AI Decisions in IoT - MDPI, accessed

February 13, 2026, https://www.mdpi.com/2624-831X/6/3/37
10. Quantitative Risk Scoring for Autonomous AI Agents: Integrating Intent, Capabilities, and Cross-Agent Interactions | by Ruchik Dave | Medium, accessed February 13, 2026, https://medium.com/@ruchikd/quantitative-risk-scoring-for-autonomous-ai-agents-integrating-intent-capabilities-and-a83a78ae9ce9
11. How AI agents are transforming finance and reporting - PwC, accessed February 13, 2026, https://www.pwc.com/us/en/services/audit-assurance/library/ai-agents-for-finance-and-reporting.html
12. Evaluating autonomous AI agents for performance, oversight, and business value - Wandb, accessed February 13, 2026, https://wandb.ai/site/articles/evaluating-autonomous-ai-agents/
13. Prevent AI Agent Hallucinations in Production Environments - StackAI, accessed February 13, 2026, https://www.stack-ai.com/insights/prevent-ai-agent-hallucinations-in-production-environments
14. Safety, Guardrails & Compliance in AI Agents: What You Must Know - CoSupport AI, accessed February 13, 2026, https://cosupport.ai/articles/safety-guardrails-compliance-ai-agents
15. Bash: Archiving 30 days or older files - Stack Overflow, accessed February 13, 2026, https://stackoverflow.com/questions/72789369/bash-archiving-30-days-or-older-files
16. Archiving repositories - GitHub Docs, accessed February 13, 2026, https://docs.github.com/en/repositories/archiving-a-github-repository/archiving-repositories
17. Managing large Git Repositories - GitHub Well-Architected, accessed February 13, 2026, https://wellarchitected.github.com/library/architecture/recommendations/scaling-git-repositories/large-git-repositories/
18. Markdown Rendering with Awk - Graham Marlow, accessed February 13, 2026, https://mgmarlow.com/words/2024-03-23-markdown-awk/
19. How to use grep or awk to extract specific block from a file - Stack Overflow, accessed February 13, 2026, https://stackoverflow.com/questions/11593569/how-to-use-grep-or-awk-to-extract-specific-block-from-a-file
20. use sed (or awk, perl etc) to identify first occurrence of a markdown title - Stack Overflow, accessed February 13, 2026, https://stackoverflow.com/questions/14300527/use-sed-or-awk-perl-etc-to-identify-first-occurrence-of-a-markdown-title
21. Awk script to generate HTML documentation from markdown text in source code comments., accessed February 13, 2026, https://gist.github.com/wernsey/eb1525876ca5519822ab48e68207970f
22. How to use awk to extract a YAML metadata block - Unix & Linux Stack Exchange,

accessed February 13, 2026, https://unix.stackexchange.com/questions/722310/how-to-use-awk-to-extract-a-yaml-metadata-block
23. AI Agent Guardrails for Secure and Compliant AI - WitnessAI, accessed February 13, 2026, https://witness.ai/blog/ai-agent-guardrails/
24. Check if a string contains valid pattern in Bash - Stack Overflow, accessed February 13, 2026, https://stackoverflow.com/questions/73119855/check-if-a-string-contains-valid-pattern-in-bash
25. Essential Guide to LLM Guardrails: Llama Guard, NeMo.. | by Sunil Rao - Medium, accessed February 13, 2026, https://medium.com/data-science-collective/essential-guide-to-llm-guardrails-llama-guard-nemo-d16ebb7cbe82